Database
(SOC-3020)

Alessandro Agostini[1,2]

aagostini@inha.ac.kr
agostini@disi.unitn.it

[1]LDKR-Group, School of Computer and Information Engineering
Inha University in Tashkent (IUT), Uzbekistan

[2]DISI, University of Trento, Italy

WEEK 6





DATABASE

SQL: Intermediate

WEEK 6

Code: W6.L1L2[LAB#3]

Outline

1. From Previous Week
2. Join Expressions
3. Set Operations
4. Additional Basic Operations
5. Aggregate Functions
6. Nested Subqueries
7. LAB #3 : Practice on SQL with MySQL



Any question from you?

...

Outline

1 From Previous Week

2 Join Expressions

3 Set Operations

4 Additional Basic Operations

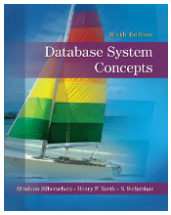
5 Aggregate Functions

6 Nested Subqueries

7 LAB #3 : Practice on SQL with MySQL

Join Expressions (*cont.*)

Ch 4, T; section 4.1.





Join Expressions

- **Join operations** (in form of join expressions) take two relations and return as a result another relation.
- **Def:** A **join** operation is a Cartesian product A x B which requires that tuples in the relations A and B match *under some condition*.
  - A join operation always specify the attributes that are present in the result of the join.
- The join operations are typically used as subquery expressions in the **from** clause.



Outer Joins

- **Def.** An extension of (*Inner, Cross*) Join (=Cartesian Product) that avoids loss of data (mismatch) when creating new information (by joining).



Natural Join: Problem

- Consider the SQL query:

select *
from *student* **natural join** *takes*;

student

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

?

takes

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	



Natural Join: Problem

- "Find all data of all students with the courses they have taken"

student

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

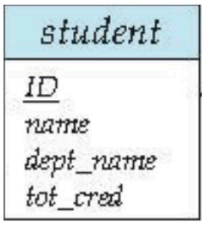
?

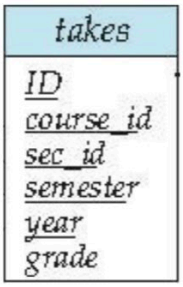
takes

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	



Inner vs Outer Joins





"Find all data of all students with the courses they have taken":

select *
from *student* **natural**
left outer join *takes*;

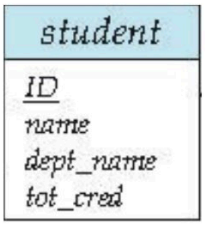
Note: the **natural join** we studied before is called **natural inner join**

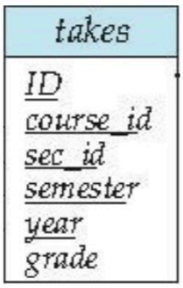
student → *takes*

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2009	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2009	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2009	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2009	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2010	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2009	A
19991	Brandt	History	80	HIS-351	1	Spring	2010	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2010	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2009	B-
45678	Levy	Physics	46	CS-101	1	Fall	2009	F
45678	Levy	Physics	46	CS-101	1	Spring	2010	B+
45678	Levy	Physics	46	CS-319	1	Spring	2010	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2009	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2009	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2010	A-
70557	Snow	Physics	0	*null*	*null*	*null*	*null*	*null*
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2009	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2010	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2009	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2009	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2010	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2009	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	201	



Inner vs Outer Joins





"Find data of all courses and of the students who have taken them":

select *
from *student* **natural**
right outer join *takes*;

Note: **right outer join** symmetric to **left outer join**

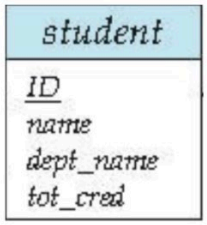
NB: **right outer join** in this example is conceptually wrong!

student → *takes*

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2009	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2009	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2009	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2009	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2010	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2009	A
19991	Brandt	History	80	HIS-351	1	Spring	2010	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2010	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2009	B-
45678	Levy	Physics	46	CS-101	1	Fall	2009	F
45678	Levy	Physics	46	CS-101	1	Spring	2010	B+
45678	Levy	Physics	46	CS-319	1	Spring	2010	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2009	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2009	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2010	A-
null	*null*	*null*	*null*	CS-190	2	Spring	2009	A
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2009	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2010	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2009	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2009	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2010	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2009	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	201	



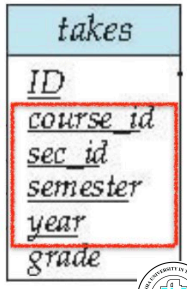
Full Outer Join



- "Find all data of all students in CS department with the courses they have taken, if any, and all course sections they have taken in Spring 2009 "

```
select *
from (select *
        from student
        where dept_name = 'Comp. Sci')
        natural full outer join
       (select *
        from takes
        where semester = 'Spring' and year = 2009);
```

note: PK for course section





Joined Relations: Summary

- **Join operations** take two relations and return as a result another relation.
- These additional operations are typically used as subquery expressions in the **from** clause
- **Join condition** – defines which tuples in the two relations match, and what attributes are present in the result of the join.
- **Join type** – defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated.

Cartesian Product →

Join types
(inner)join
left outer join
right outer join
full outer join

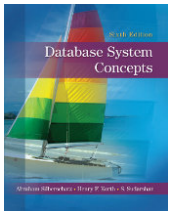
Join Conditions
natural
on <predicate>
using $(A_1, A_1, \ldots, A_n)$

Outline

1. From Previous Week
2. Join Expressions
3. Set Operations
4. Additional Basic Operations
5. Aggregate Functions
6. Nested Subqueries
7. LAB #3 : Practice on SQL with MySQL

Set Operations in SQL

Ch 3, T; section 3.5





Set Operations — Introduction

- The SQL operations **union**, **intersect** and **except** (also called **minus**) operate on relations (sets);
  - correspond to the mathematical (set-theoretical) operations ∪, ∩, and − (or / set difference).
- We shall now construct queries involving the **union**, **intersect**, and **except** operations over two relations (sets) *r1* and *r2* defined as in the next slide.





Set Operations (*cont.*)

- *r1*: the relation resulting from the SQL query:

select *course_id*
from *section*
where *semester* = 'Fall' [**and** *year* = '2009'];

section:

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

r1:

course_id
CS-101
CS-347
PHY-101

optional
(in this case)





Set Operations (*cont.*)

- *r2*: the relation resulting from the SQL query:

```
select course_id
from section
where semester = 'Spring' and year = '2010';
```

section:

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

r2:

course_id
CS-101
CS-315
CS-319
CS-319
FIN-201
HIS-351
MU-199





Set Operations: "union"

- "Find all course id's of courses taught either in Fall 2009 or Spring 2010 (or both)"

```
(select course_id
from section
where semester = 'Fall' and year = '2009')
union [all]
(select course_id
from section
where semester = 'Spring' and year = '2010');
```

to retain all duplicates

r1:

course_id
CS-101
CS-347
PHY-101

r2:

course_id
CS-101
CS-315
CS-319
CS-319
FIN-201
HIS-351
MU-199

r1 ∪ r2:

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101
CS-101





Remark: Equivalent (easier) query

- "Find all course id's of courses taught either in Fall 2009 or Spring 2010 (or both)"

```
select course_id
from section
where (semester = 'Fall' and year = '2009') or
      (semester = 'Spring' and year = '2010');
```

- Note: to express the "and" we need a set operation!

(*see the next slide*)





Set Operations: "intersect"

- "Find all course id's of courses taught in Fall 2009 and Spring 2010"

to retain all duplicates

```
(select course_id
from section
where semester = 'Fall' and year = '2009')
intersect [all]
(select course_id
from section
where semester = 'Spring' and year = '2010');
```

r1:

course_id
CS-101
CS-347
PHY-101
CS-101

r2:

course_id
CS-101
CS-315
CS-319
CS-319
FIN-201
HIS-351
MU-199
CS-101

r1 ∩ r2:

course_id
CS-101
CS-101





Set Operations: "except" ("minus")

- "Find all course id's of courses taught in Fall 2009 but not in Spring 2010"

```
(select course_id
from section
where semester = 'Fall' and year = '2009')
except [all]
(select course_id
from section
where semester = 'Spring' and year = '2010');
```

to retain all duplicates

r1:

course_id
CS-101
CS-347
PHY-101
CS-101

r2:

course_id
CS-101
CS-315
CS-319
CS-319
FIN-201
HIS-351
MU-199

r1 − r2:

course_id
CS-347
PHY-101
CS-101



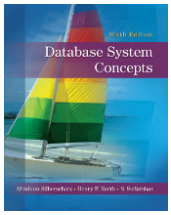
Outline

1. From Previous Week
2. Join Expressions
3. Set Operations
4. Additional Basic Operations
5. Aggregate Functions
6. Nested Subqueries
7. LAB #3 : Practice on SQL with MySQL

Additional * Basic Operations

Ch 3, T; section 3.4

* "Additional" = Out of the scope of Data Model



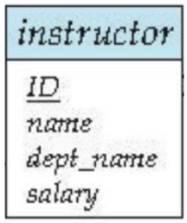


Remember: "select" clause + expressions

- The **select** clause can contain arithmetic expressions involving the operation, +, –, ∗, and /, and operating on constants or attributes of tuples.
  - **Example**:

    select *ID, name, dept_name, salary / 12*
    from *instructor*;

  would return a relation that is the same as the *instructor*, except that the value of *salary* is divided by 12, so resulting in the monthly salary (assuming *salary* is annual).



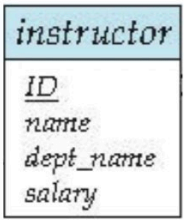
- We can rename "*salary / 12*" using the **as** clause:

  select *ID, name, dept_name salary / 12* **as** *monthly_salary*;



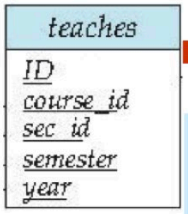


Renaming — The "as" Clause



- "Find instructors' name and course id of courses they taught for all instructors in the Biology department who have taught some course"

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID* **and** *dept_name* = 'Biology';



- Renaming (the "**as**" clause, ref. sec. 3.4.1):

select *name, course_id*
from *instructor* **as** *I*, *teaches* **as** *T*
where *I.ID = T.ID* **and** *dept_name* = 'Biology';

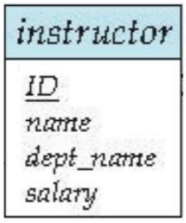




More on Renaming

- "Find the salaries of all instructors that are less than the largest salary of any (some) instructor in the Biology department"

- In the query we compare tuples in the same relation (*instructor*)



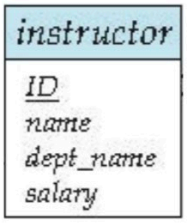




More on Renaming

- "Find the salaries of all instructors that are less than the largest salary of any (some) instructor in the Biology department"
- In the query we compare tuples in the same relation (*instructor*)

select distinct *T.salary*
from *instructor* **as** *T*, *instructor* **as** *S*
where *T.salary* < *S.salary* **and** *S.dept_name* = 'Biology';







Attribute Specification - The "*" Operation

- "Find the Cartesian product *instructor* x *teaches*"

select *
from *instructor, teaches;*





Ordering the Display of Tuples

- List in alphabetic order the names of all instructors:

```
select distinct name
from instructor
order by name;
```

We may specify **desc** for descending order or **asc** for ascending order (default), for each attribute.

- **Example**: **order by** *name* **desc**
- Can sort on multiple attributes
  - **Example**: **order by** *dept_name, name*





Another Example

- "List the entire *instructor* relation in descending order of *salary*. If some instructors have the same salary, list them in ascending order by name".

```
select *
from instructor
order by salary desc, name asc;
```





String Operations - The "like" Clause

- SQL includes a string-matching operator for comparisons on character strings. The operator **like** uses patterns that are described using two special characters:
  - percent (%). The % character matches any substring.
  - underscore (_). The _ character matches any character.
- **Example**: "Find the names of all instructors whose name includes the substring 'dar'".

```
select name
from instructor
where name like '%dar%'
```

- Match the string "100%"

```
like '100 \%' escape '\'
```

in that above we use backslash (\) as the escape character.





The "between" Clause

- SQL includes a **between** comparison operator
- "Find the names of all instructors with salary between $90,000 and $100,000"

```
select name
from instructor
where salary between 90000 and 100000;
```





"where" Clause: Comparison on Tuples

- "Find instructors' name and course id of courses they taught for all instructors in the Biology department who have taught some course"

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID and dept_name = 'Biology';
```

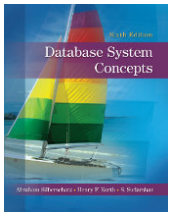
- Rewriting the above query by tuple comparison operator:

```
select name, course_id
from instructor, teaches
where (instructor.ID, dept_name) = (teaches.ID, 'Biology');
```



Null Values

Ch 3, T; section 3.6





Null Values

- It is possible for tuples to have a null value, denoted by *null*, for some of their attributes
- *null* means unknown value or a value does not exist.
- Result of arithmetic expressions involving *null* is *null*
  - **Example**: "5 + *null*" returns *null*.

- Predicate **is null** can be used to check unknown data.
  - "Find all instructors whose salary is unknown"

```
select name
from instructor
where salary is null
```





Null Values and Three Valued Logic

- Three values : *true*, *false*, *unknown*
- Any comparison with *null* returns *unknown*
  - **Example**: *5 < null or null <> null or null = null*
- Three-valued logic using the value *unknown*:
  - OR: (*unknown* **or** *true*) = *true*,
    (*unknown* **or** *false*) = *unknown*
    (*unknown* **or** *unknown*) = *unknown*
  - AND: (*true* **and** *unknown*) = *unknown*,
    (*false* **and** *unknown*) = *false*,
    (*unknown* **and** *unknown*) = *unknown*
  - NOT: (**not** *unknown*) = *unknown*
- If predicate *P* evaluates to either *false* or *unknown* in **where** clause for tuple t, then t is not added to the result

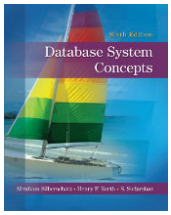


Outline

1. From Previous Week
2. Join Expressions
3. Set Operations
4. Additional Basic Operations
5. Aggregate Functions
6. Nested Subqueries
7. LAB #3 : Practice on SQL with MySQL

Aggregate Functions

Ch 3, T; section 3.7





Aggregate Functions

- **Def**: Aggregate functions are functions that operate on a collection (a set or a multi-set) of values of an attribute of a relation, and return a single value.
- In SQL standard (cf. textbook), these functions are:

avg():	average value
min():	minimum value
max():	maximum value
sum():	sum of values
count():	number of values





Aggregate Functions (*cont.*)

- "Find the average salary of instructors in the Computer Science department"

gives name "*avg_salary*" to the resulting relation's attribute

```
select avg (salary) as avg_salary
from instructor
where dept_name= 'Comp. Sci.';
```

instructor:

ID	*name*	*dept_name*	*salary*
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000





Aggregate Functions (*cont.*)

- "Find the average salary of instructors in the Computer Science department"

gives name "*avg_salary*" to the resulting relation's attribute

```
select avg (salary) as avg_salary
from instructor
where dept_name= 'Comp. Sci.';
```

instructor:

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

Remark:
Don't use renaming for no precise purpose of the query expression.





Aggregate Functions (*cont.*)

- "Find the total number of instructors' ID of instructors who teach or have taught a course in the Spring 2010 semester"

```
select count (distinct ID)
from teaches
where semester = 'Spring' and year = 2010;
```

eliminates duplicates before aggregating

teaches:

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009





Aggregate Functions (*cont.*)

- "Find the number of *all* tuples in the *course* relation"

?

course:

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4





Aggregate Functions (*cont.*)

- "Find the number of all tuples in the *course* relation"

select count (*)
from *course*;

course:

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4





Aggregate Functions – "group by"

- "Find the average salary of instructors in each department"

instructor:

departments- grouping

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000





Aggregate Functions – "group by"

- "Find the average salary of instructors in each department"

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name;
```

instructor:

departments- grouping

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000





Erroneous Query with "group by"

- "Find the average salary of instructors in each department"

```
select dept_name,ID, avg (salary) as avg_salary
from instructor
group by dept_name;
```

instructor:

ID	*name*	*dept_name*	*salary*
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

departments- grouping

dept_name	*avg_salary*
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000





Aggregate Functions – "group by"

- "Find the average salary of all instructors"

```
select avg (salary) as avg_salary
from instructor;
```

instructor:

ID	*name*	*dept_name*	*salary*
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

no grouping = one group

avg_salary
72000
77333
80000
85000
61000
40000
91000
74833

(898000/12)





Exercise

- Write a SQL query that takes in input *instructor* and *teaches* and produces in output table below.

instructor:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Read it:
"instructor count"

dept_name	instr_count
Comp. Sci.	3
Finance	1
History	1
Music	1

teaches:

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	200
83821	CS-319	2	Spring	201
98345	EE-181	1	Spring	200





Exercise (*cont.*)

- "Find the number of instructors in each department who teach a course in the Spring 2010 semester"

instructor:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dept_name	instr_count
Comp. Sci.	3
Finance	1
History	1
Music	1

teaches:

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	200
83821	CS-319	2	Spring	201
98345	EE-181	1	Spring	200



Exercise (*cont.*)

- "Find the number of instructors in each department who teach a course in the Spring 2010 semester"

select *dept_name*, **count** (**distinct** *ID*) **as** *instr_count*
from *instructor* **natural join** *teaches*
where *semester* = 'Spring' **and** *year* = *2010*
group by *dept_name*;

dept_name	*instr_count*
Comp. Sci.	3
Finance	1
History	1
Music	1

Observe

instructor:

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

teaches:

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	200
83821	CS-319	2	Spring	201
98345	EE-181	1	Spring	200





Aggregate Functions – "having"

- Useful to state a condition that applies to groups of tuples rather than to all tuples (as we do with **where** clause).
- "Find the names and average salaries of all departments whose average salary is greater than $42000"

instructor:

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dept_name	*avg_salary*
Physics	91000
Elec. Eng.	80000
Finance	85000
Comp. Sci.	77333
Biology	72000
History	61000





Aggregate Functions – "having"

- Useful to state a condition that applies to groups of tuples rather than to all tuples (as we do with **where** clause).

instructor
ID
name
dept_name
salary

- "Find the names and average salaries of all departments whose average salary is greater than $42000"

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name
having avg_salary > 42000;
```

dept_name	avg_salary
Physics	91000
Elec. Eng.	80000
Finance	85000
Comp. Sci.	77333
Biology	72000
History	61000





Aggregate Functions – "having"+"where"

- "For each course section offered in 2009, find the average total credits (*tot_cred*) of all students enrolled in that section, if the section had at least 2 students"

section:

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

= Primary Key

student:

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110

takes:

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+

56
46
54
38
98





Aggregate Functions – "having"+"where"

- "For each course section offered in 2009, find the average total credits (*tot_cred*) of all students enrolled in that section, if the section had at least 2 students"

```
select course_id, semester, year, sec_id, avg(tot_cred) as
avg_tot_cred
from takes natural join student
where year = 2009
group by course_id, semester, year, sec_id
having count (ID) >= 2;
```

student:

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110

takes:

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+

course_id	sec_id	semester	year	avg_tot_cred
CS-347	1	Fall	2009	(102+32)/2
...	...	...	...	...

Outline

1 From Previous Week

2 Join Expressions

3 Set Operations

4 Additional Basic Operations

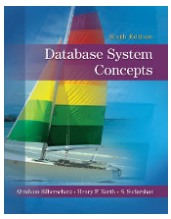
5 Aggregate Functions

6 Nested Subqueries

7 LAB #3 : Practice on SQL with MySQL

Nested Subqueries

Ch 3, T; section 3.8





Nested Subqueries

- **Def**: A **subquery** is a **select-from-where** expression (so, a subquery *is* a SQL query!) that is nested within another query.
- In a SQL query

  select $A_1, A_2, ..., A_n$
  from $r_1, r_2, ..., r_m$
  where P ;

  - A_i can be replaced be a subquery that generates a single value.
  - r_i can be replaced by any valid subquery.
  - P can be replaced with an expression of the form:

    Attribute <operation> (**subquery**)

    where <operation> is to be defined (*see later*).





Nested Subqueries (*cont.*)

- As the nesting can be done in the following SQL query

select $A_1, A_2, ..., A_n$
from $r_1, r_2, ..., r_m$
where P ;

we explain in turn:

- subqueries in the **where** clause;
- subqueries in the **from** clause;
- subqueries in the **select** clause.



Nested Subqueries in "where"

Attribute <operation> (**subquery**)



Subqueries in the "where" Clause

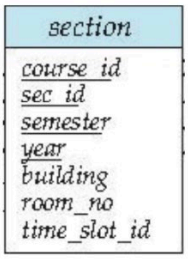
- A common use of subqueries is to perform tests:
  - for set membership;
  - for set comparisons;
  - for set cardinality (e.g., empty relations).



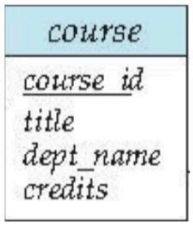


Set Membership

- "Find course id's of courses offered in Fall 2009 and Spring 2010"




```
(select course_id
   from section
   where semester = 'Fall' and year = '2009')
intersect
(select course_id
   from section
   where semester = 'Spring' and year = '2010');
```







Set Membership

- "Find course id's of courses offered in Fall 2009 and Spring 2010"
- An alternative approach in SQL by using subqueries is:

```
select distinct course_id
from section as S
where semester = 'Fall' and year= 2009 and
      course_id in (select course_id
                    from section as T
                    where semester = 'Spring' and year= 2010
                          and T.course_id = S.course_id);
```

correlation name

Observe: this relates to "**intersect**" (that by default automatically eliminates duplicates; to retain use "**intersect all**")





Set Membership

- "Find course id's of courses offered in Fall 2009 but not in Spring 2010"

section
course_id
sec_id
semester
year
building
room_no
time_slot_id

```
(select course_id
   from section
   where semester = 'Fall' and year = '2009')
except
(select course_id
   from section
   where semester = 'Spring' and year = '2010');
```

course
course_id
title
dept_name
credits





Set Membership

- "Find course id's of courses offered in Fall 2009 but not in Spring 2010"

- An alternative approach in SQL by using subqueries is:

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
      course_id not in (select course_id
                        from section
                        where semester = 'Spring' and year= 2010);
```

Observe: this relates to "**except**" (that by default automatically eliminates duplicates; to retain use "**except all**")





Enumerated Set Membership

- "Find names of instructors whose name is neither Mozart nor Einstein"

```
select distinct name
from instructor
where name not in ('Mozart', 'Einstein');
```

instructor
ID
name
dept_name
salary

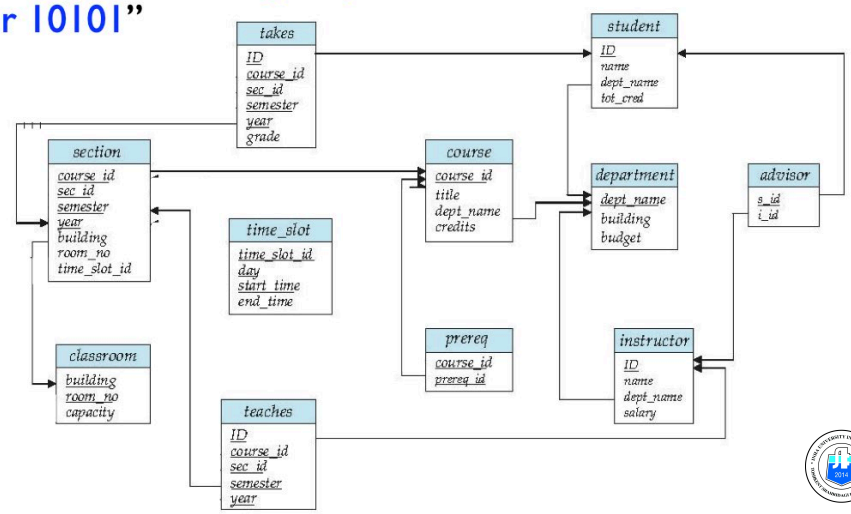
A set where elements are explicitly listed is called enumerated set





Multi-Attribute Set Membership

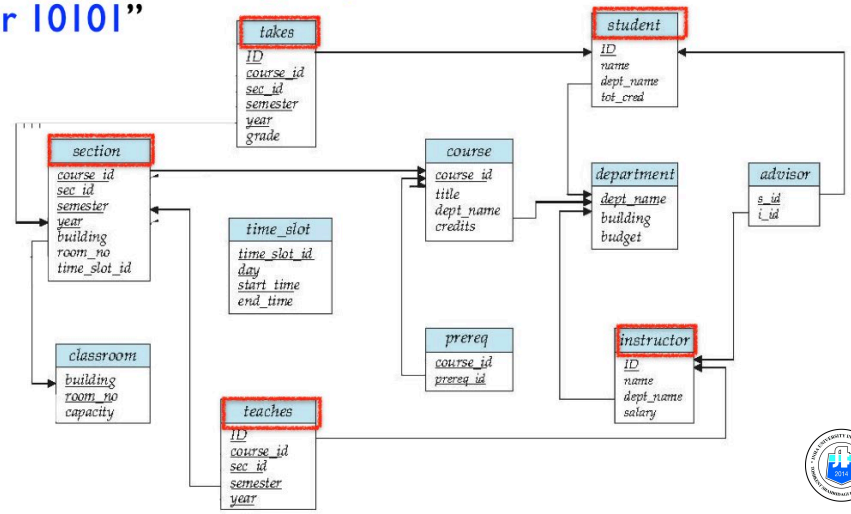
- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"





Multi-Attribute Set Membership

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"





Multi-Attribute Set Membership

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"

```
select count (distinct S.ID)
  from student as S, takes, section, teaches, instructor as I
  where I.ID = '10101' and …;
```

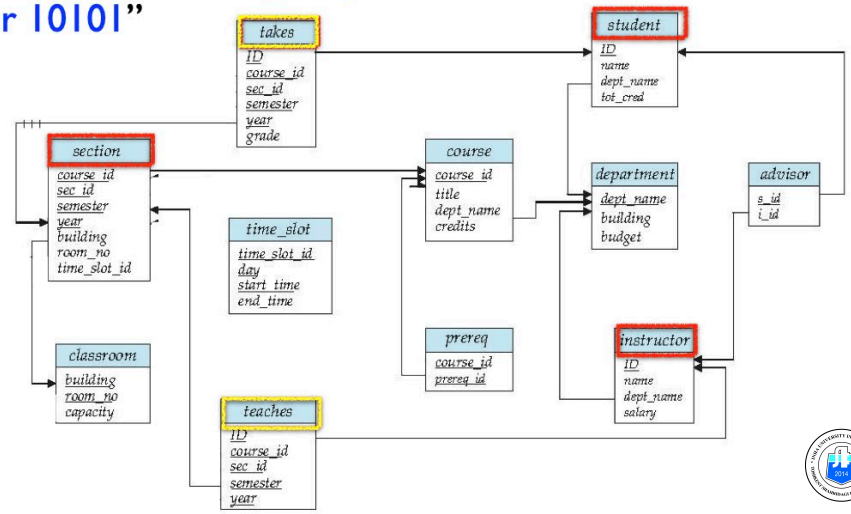
where "…" means that you have to control all "matching attributes"





Multi-Attribute Set Membership

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"



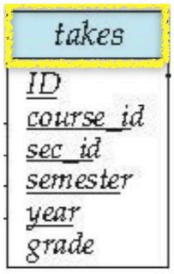


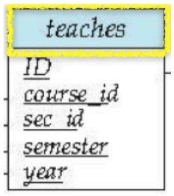


Multi-Attribute Set Membership

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"

```
select count (distinct S.ID)
  from takes as S join teaches as I using (course_id, sec_id,
                                           semester, year)
  where I.ID = '10101';
```







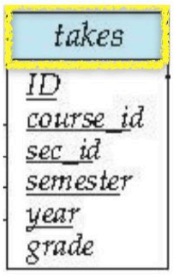


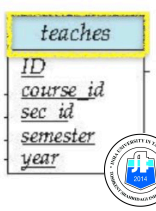
Multi-Attribute Set Membership

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"

- An alternative approach in SQL by using subqueries is:

```
select count (distinct ID)
from takes
where (course_id, sec_id, semester, year) in
                (select course_id, sec_id, semester, year
                 from teaches
                 where teaches.ID= 10101);
```









Multi-Attribute Set Membership

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"

subquery

teaches:

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

count(distinct *S.ID*) = 6

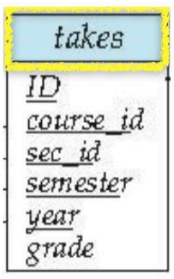
takes:

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	null



Multi-Attribute Set Membership: Remark 1

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"




```
select count (distinct ID)
from takes
where (course_id, sec_id, semester, year) in
                (select course_id, sec_id, semester, year
                 from teaches
                 where teaches.ID= 10101);
```





Multi-Attribute Set Membership: Remark 2

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"
- This query represents a different table than before:

```
select count (distinct ID)
from takes
where (course_id, sec_id, semester, year) in
                (select course_id, sec_id, semester, year
                 from teaches
                 where teaches.ID= 10101);
```





Remark 2 (cont.)

- "Find the total number of (distinct) students who have taken a course section taught by the instructor with ID number 10101"

subquery

teaches:

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

count(distinct S.ID) = 6+6!

takes:

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	null





Subqueries in the "where" Clause (*cont.*)

- A common use of subqueries is to perform tests:
  - for set membership;
  - for set comparisons;
  - for set cardinality (e.g., empty relations).





Set Comparison – The "some" Clause

- "Find names of all instructors with salary greater than that of some (at least one) instructor in the Biology department"

?

instructor
ID
name
dept_name
salary





A solution we have seen...

- "Find names of all instructors with salary greater than that of some (at least one) instructor in the Biology department"

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Biology';
```

instructor
- *ID*
- *name*
- *dept_name*
- *salary*





Set Comparison – The "some" Clause

- "Find names of all instructors with salary greater than that of some (at least one) instructor in the Biology department"
- Same previous query by using the "**> some**" clause:

Means: "greater than at least one"

```
select name
from instructor
where salary > some (select salary
                     from instructor
                     where dept_name = 'Biology');
```

- Also: **< some, ≤ some, = some, ≥ some, <> some**



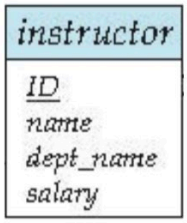


Set Comparison – The "all" Clause

- "Find names of all instructors with salary greater than that of all the instructors in the Biology department"

Means: "greater than all"

```
select name
from instructor
where salary > all (select salary
                    from instructor
                    where dept_name = 'Biology');
```



- Also: < **all**, ≤ **all**, = **all**, ≥ **all**, <> **all**

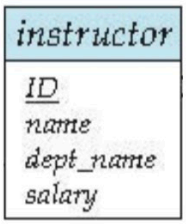




More on Set Comparison

- "Find the departments that have the highest average salary"

?







More on Set Comparison - Example

- "Find department's names with highest average salary"
- First, we find the average salary for each department:

```
select avg (salary)
from instructor
group by dept_name;
```

- Then, we use the query above as subquery:

```
select dept_name
from instructor
group by dept_name
having avg (salary) >= all (select avg (salary)
                            from instructor
                            group by dept_name);
```





More on Set Comparison - Example

- "Find department's names with highest average salary"
- First, we find the average salary for each department:

```
select avg (salary)
from instructor
group by dept_name;
```

- Then, we use the query above as subquery:

```
select dept_name, [ avg (salary) ]
from instructor
group by dept_name
having avg (salary) >= all (select avg (salary)
                            from instructor
                            group by dept_name);
```





Subqueries in the "where" Clause (*cont.*)

- A common use of subqueries is to perform tests:
  - for set membership;
  - for set comparisons;
  - for set cardinality (e.g., empty relations).





Empty Relations – The "exists" Clause

- The **exists** clause returns the value **true** if the argument subquery is nonempty.
  - **exists** *r* is true $\Leftrightarrow$ $r \neq \emptyset$
  - **not exists** *r* is true $\Leftrightarrow$ $r = \emptyset$





(reminder) Two alternative solutions

- "Find course id's of courses offered in Fall 2009 and Spring 2010"
- Two SQL queries we have seen so far for it:

```
(select course_id
 from section
 where semester = 'Fall' and year = '2009')
intersect
(select course_id
 from section
 where semester = 'Spring' and year = '2010');


select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
      course_id in (select course_id
                    from section
                    where semester = 'Spring' and year= 2010);
```





The "exist" Clause (*cont.*)

- "Find course id's of courses offered in Fall 2009 and Spring 2010"
- A third approach in SQL by using a subquery:

```
select course_id
from section as S
where semester = 'Fall' and year = 2009 and
      exists (select *
              from section as T
              where semester = 'Spring' and year= 2010
                    and S.course_id = T.course_id);
```

correlation name

called: correlated subquery





Remark

- Is the following SQL query equivalent to the previous SQL query? Why / Why not?

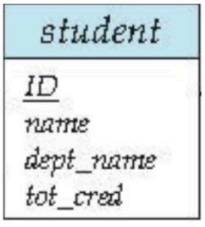
```
select course_id
from section as S
where semester = 'Fall' and year = 2009 and
        exists (select *
                from section as T
                where semester = 'Spring' and year= 2010
                        and S.course_id = T.course_id);
```

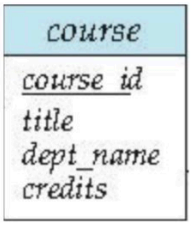


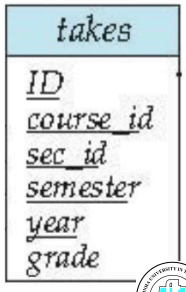


The "not exist" Clause

- "Find ID and name of all students who have taken all courses offered in the Biology department"



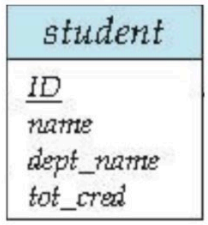








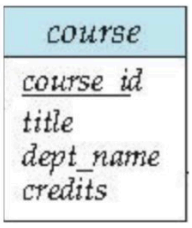
The "not exist" Clause

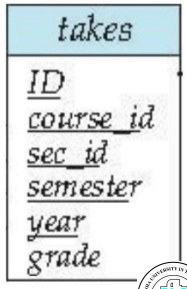


- "Find ID and name of all students who have taken all courses offered in the Biology department"

```
select ID, name
from student as S
where not exists ( ( select course_id
                     from course
                     where dept_name = 'Biology' )
                   except
                   ( select T.course_id
                     from takes as T
                     where S.ID = T.ID ) );
```

correlation name





- courses offered in Biology dept.
- courses taken by students



Nested Subqueries in "from"

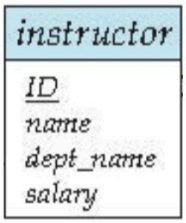
Any valid **subquery** is accepted



Example

- "Find the names and average salaries of all departments whose average salary is greater than $42000"

?









A solution we have seen...

- "Find the names and average salary of all departments whose average salary is greater than $42000"

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name
having avg_salary > 42000;
```

dept_name	avg_salary
Physics	91000
Elec. Eng.	80000
Finance	85000
Comp. Sci.	77333
Biology	72000
History	61000





Subqueries in the "from" Clause (*cont.*)

- "Find the names and average salary of all departments whose average salary is greater than $42000"

dept_name	*avg_salary*
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

- Same query by using a subquery is:

```
select dept_name, avg_salary
from (select dept_name, avg (salary) as avg_salary
      from instructor
      group by dept_name)
where avg_salary > 42000;
```

Note: "**having**" not needed clause "**where**" used indeed





Subqueries in the "from" Clause

- "Find the maximum across all departments of the total salary at each department"

?

instructor
ID
name
dept_name
salary

department
dept_name
building
budget

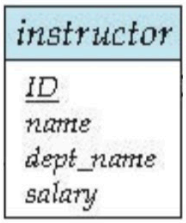




Subqueries in the "from" Clause (*cont.*)

- "Find the maximum across all departments of the total salary at each department"

```
select max (total_salary)
from (select dept_name, sum (salary) as tot_salary
      from instructor
      group by dept_name);
```



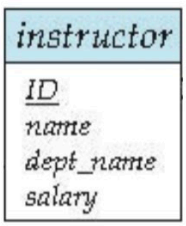




Subqueries in the "from" Clause (*cont.*)

- "Find the maximum across all departments of the total salary at each department"

```
select max (total_salary)
from (select dept_name, sum (salary)
      from instructor
      group by dept_name) as dept_total (dept_name, tot_salary);
```



named subquery: *not always* supported in SQL implementations

(see if your MySQL version supports it)



Nested Subqueries in “select”

Subqueries with result a single value (“**Scalar Queries**”)



Scalar Subquery

- Scalar subquery is one which is used where a single value (i.e., one and only one row!) is expected.
- “List all departments along with the number of instructors in each department”

```
select dept_name,
        (select count(*)
         from instructor
         where department.dept_name = instructor.dept_name) as
num_instructors
 from department;
```

note: **count**(*) produces one single value!

correlation variable (cf. p. 98, T)

- Runtime error if subquery returns more than one result tuple. Also: note the naming of the subquery (optional here).



Outline

1. From Previous Week
2. Join Expressions
3. Set Operations
4. Additional Basic Operations
5. Aggregate Functions
6. Nested Subqueries
7. LAB #3 : Practice on SQL with MySQL

LAB #3: Practice on SQL

* Participation checked (2-3 members per working group)

In class only (no slides provided).



Summary

1 From Previous Week

2 Join Expressions

3 Set Operations

4 Additional Basic Operations

5 Aggregate Functions

6 Nested Subqueries

7 LAB #3 : Practice on SQL with MySQL



DATABASE

End of Slides

WEEK 6

Code: W6.L1L2[LAB#3]